

NER Fine-tuning

NER 모델 학습과 결과 추론에 대한 코드가 담긴 리포지토리

[훈련에 앞서 필요한 과정]

1. 환경구축

코드 실행에 필요한 모든 패키지를 콘다를 통해 다운받는다. 아래의 코드를 실행시키면 된다. **yml** 파일이 이 폴더의 루트 디렉토리에 있다는 것을 주의해야 한다.

```
conda env create -f environment.yml
```

2. 데이터 전처리

훈련을 하기 전에, 데이터 전처리 과정이 필요하다. 데이터 전처리의 최종 목표는 가지고 있는 데이터들의 포맷을 huggingface에 규정되어 있는 NER 데이터 포맷으로 바꾸는 것이다. 기본적으로, 데이터는 tokens와 ner_tags의 리스트로 구성되어 있어야 한다. ([CONLL-2003 dataset](#)과 유사한 형태)

데이터 전처리는 세 단계로 이루어진다.

(1) json 파일들의 포맷을 수정

이 단계에서는 토큰들을 분리하고 NER tag 라벨이 NER tag ID로 구성되도록 수정하는 것이다. 실행에 앞서 **preprocess_dataset_json.py** script에서 경로 설정이 제대로 되어 있는지 확인해야 한다.

```
parent_path = Path(# insert the data path here)
```

그 후 아래의 코드를 실행하면 된다.

```
python preprocess_dataset_json.py
```

주의할 점은, 이 코드는 원본 json파일의 형태를 바꾼다. 따라서 오류나 예외적 상황을 대비하여

백업 파일을 가지고 있는게 좋다.

(2) 각 폴더 안의 json파일들을 합쳐 하나의 jsonl파일로 만든다.

jsonl 파일은 폴더별로 하나씩 생성된다.

우리는 수백만개의 데이터를 처리하기 때문에 모든 데이터를 읽어오는 것보다 각 폴더당 하나의 jsonl파일을 읽어온다면 더 빠르게 읽어올 수 있을 것이다. json파일을 합치기 위해서 아래의 코드를 실행하면 된다.

```
python merge_json.py
```

트위터 데이터의 경우, jsonl 파일은 3개의 json 파일들의 묶음으로 이루어져 있다. 기사 데이터의 경우는 모든 json 파일이 하나의 jsonl 파일로 이루어져 있다.

이번에도 코드 실행 전에 script 상 경로 설정이 제대로 이루어져 있는지 꼭 확인해야 한다.

```
parent_path = Path(# insert the data path here)
```

(3) jsonl 파일을 읽어와서, 데이터를 train, validation, test 셋으로 나눈다.

이 단계에서는 데이터를 나누고 데이터를 읽어오는 시간을 줄이기 위해 나누어진 데이터를 huggingface 리포지토리에 업로드한다.

코드 실행 전에, **preprocess_dataset_jsonl.py** 파일에서 폴더 경로와 데이터 파일명을 수정했는지 확인해야 한다.

```
parent_path = Path(# insert the data path here)
```

그 후, 코드 script 에서 데이터 리포지토리 이름을 변경한다.

```
train_test_valid_dataset.push_to_hub("arspraxia/arspraxia_ner", private=True)
```

위의 코드를 실행할 경우, 데이터는 "arspraxia" huggingface 계정에 "arspraxia_ner"이라는 데이터셋 명으로 업로드된다. 데이터셋은 **private = True**로 지정했기 때문에 private 상태로 업로드된다. 즉, huggingface token key를 통해서만 데이터셋에 접근 가능하다. 따라서 데이터를 업로드하기 전에 access token을 가지고 있는지 확인해야 한다.

위의 모든 것을 완료했다면, 이 코드를 실행시키면 된다.

```
python preprocess_dataset_jsonl.py
```

[훈련]

데이터 전처리가 끝났다면, 아래의 코드를 활용해 훈련을 진행할 수 있다.

```
bash run_training.sh
```

주의할 점은, 코드를 실행하기 전 코드 script 안에 Huggingface token, wandb 프로젝트 명, 데이터셋 명(**dataset_name**), 모델 저장 경로(**output_dir**)와 같은 파라미터들을 올바르게 작성 했는지 확인해야 한다. 또한 훈련에 GPU를 사용한다면 **CUDA_VISIBLE_DEVICES** 파라미터를 수정해야 한다.

훈련 로그는 wandb에 기록된다. 따라서 [Wandb](#) 계정을 미리 생성해 놓아야 한다.

[추론]

이제부터 모델이 데이터를 추론하고 예측값을 얻어내는 과정에 대해 설명한다.

(1) 환경 구축

기본적으로 모델을 로드하기 위해 필요한 핵심 라이브러리 두 가지가 있다. Pytorch와 Huggingface 라이브러리이다. 가상 환경 사용을 위해 conda 사용을 추천한다.

먼저, Python 3.8 버전을 갖는 가상환경을 만든다. 그 후 가상환경을 실행시킨다.

```
conda create -n ner python=3.8
```

```
conda activate ner
```

그 후, Pytorch GPU 버전을 설치한다.

```
conda install pytorch==2.0.1 pytorch-cuda=11.7 -c pytorch -c nvidia
```

마지막으로 Huggingface를 설치한다.

```
pip install transformers==4.33.1
```

(2) 모델 다운로드

이 [링크](#)에서 사전 훈련된 모델을 다운받을 수 있다.

(3) 모델 로드

(참고 : 전체 예시 코드를 inference.ipynb 파일에서 확인할 수 있다.)

먼저, 사용하고자 하는 GPU를 설정한다.

```
!export CUDA_VISIBLE_DEVICES=1
```

필요한 패키지를 import한다.

```
from transformers import pipeline
```

이전에 [(2) 모델 다운로드] 에서 다운받은 모델의 경로를 **model_path**에 설정하고 pipeline 패키지를 사용해서 모델을 로드한다.

```
model_path = # set the model path here
```

```
classifier = pipeline("ner", model=model_path, aggregation_strategy="simple")
```

그 후, classifier 오브젝트를 호출함으로써 데이터를 예측하고 결과를 확인할 수 있다.

```
# The input_text can be in a form of string or list
```

```
input_text = "Drama ' High School Love On ' telah merilis beberapa potongan
```

gambar yang menampilkan ChoA Crayon Pop dalam balutan seragam sekolah ."

classifier(input_text)